

Tech Challenge — Fase 1

Pós Tech SOAT — Sistema Integrado de Atendimento e Execução de Serviços (Oficina Mecânica)

Nome do grupo: Lucas e José

Data de entrega: 27/04/2026

1. Participantes

Nome completo	Username no Discord
José do Egito Gomes da Silva Filho	Jose E G S Filho - RM370389
Lucas Karisson Ferreira Moreira	Lucas Moreira - RM372752

2. Links

Repositório (privado)	https://github.com/DevEgF/soat-tech-challenge-oficina Acesso liberado para o usuário <code>soat-architecture</code> .
Documentação DDD (Miro)	Abrir board no Miro Event Storming completo, Linguagem Ubíqua e Swimlane da OS. Versão textual em <code>oficina/docs/doc.md</code> .
Vídeo de demonstração	<preencher link do vídeo (até 15 min)>
Relatório de vulnerabilidades (fonte)	<code>oficina/docs/relatorio-vulnerabilidades.md</code> (consolidado abaixo, seção 4)
Coleção Postman	<code>oficina/docs/postman/Oficina.postman_collection.json</code>

3. Visão geral da entrega

MVP do back-end de uma oficina mecânica em **Kotlin + Spring Boot** (Java 17), monolito em camadas (domain / application / infrastructure / presentation), com PostgreSQL em Docker (H2 para dev local), JWT (HS256), Swagger/OpenAPI, Flyway, Docker e docker-compose. Cobertura JaCoCo configurada com mínimo de 80% nos pacotes `domain` e `application` (DTOs excluídos).

Atendimento aos critérios obrigatórios

Critério	Status	Evidência
Identificação por CPF/CNPJ (com dígito verificador)	OK	<code>domain/model/TaxDocument.kt</code>
Cadastro de veículo (placa legacy + Mercosul, marca, modelo, ano)	OK	<code>domain/model/Vehicle.kt</code> , <code>LicensePlate.kt</code>
Inclusão de serviços e peças com orçamento automático	OK	<code>WorkOrder.recalculateQuote()</code>

Critério	Status	Evidência
Envio do orçamento ao cliente para aprovação	OK	<code>WorkOrder.sendQuoteToCustomer</code>
Status: Recebida, Em diagnóstico, Aguardando aprovação, Em execução, Finalizada, Entregue	OK	<code>WorkOrderStatus.kt</code> (+ estados internos <code>PENDING_INTERNAL_APPROVAL</code> , <code>AWAITING_PARTS_RELEASE</code> , <code>CANCELLED</code>)
Transição automática conforme ações	OK	State machine com <code>requireStatus</code> em <code>WorkOrder.kt</code>
Consulta pública via API pelo cliente	OK	<code>PublicWorkOrderController</code> — <code>/api/public/os/acompanhar</code>
CRUD de clientes / veículos / serviços / peças (com estoque)	OK	Controllers <code>Admin*Controller.kt</code>
Listagem e detalhamento de OS	OK	<code>AdminWorkOrderController</code> / <code>AttendantWorkOrderController</code>
Tempo médio de execução por serviço	OK	<code>AdminMetricsController</code> + <code>MetricsServiceAdapter</code>
JWT em APIs administrativas	OK	<code>SecurityConfiguration</code> , <code>JwtIssuerService</code>
Validação CPF/CNPJ e placa	OK	Value objects <code>TaxDocument</code> e <code>LicensePlate</code>
Testes unitários e de integração	OK	<code>OrdemServicoFlowIntegrationTest</code> , <code>SwimlaneFlowsIntegrationTest</code> , <code>CurlyTestsDocumentedFlowIntegrationTest</code>
Back-end monolítico em camadas	OK	Estrutura <code>domain</code> / <code>application</code> / <code>infrastructure</code> / <code>presentation</code>
Banco com justificativa	OK	PostgreSQL (Docker) + H2 (dev) — README seção “Escolha do banco de dados”
APIs RESTful documentadas	OK	springdoc-openapi — <code>/swagger-ui.html</code>
Dockerfile + docker-compose	OK	Multi-stage com Gradle 9 / Temurin 17, Postgres 16-alpine, healthchecks
Cobertura ≥ 80% nos domínios críticos	OK	JaCoCo <code>jacocoTestCoverageVerification</code> configurado em <code>build.gradle.kts</code>
README.md explicativo	OK	Stack, fluxo, usuários demo, execução local e Docker
Repositório privado com acesso <code>soat-architecture</code>	OK	GitHub privado — convite enviado

4. Relatório de análise de vulnerabilidades

4.1 Escopo

Repositório	oficina (Spring Boot 4.1.0-SNAPSHOT / Kotlin 2.3.20 / Java 17)
Commit avaliado	83650ad
Imagem Docker	oficina-app:scan (build local a partir do Dockerfile da raiz)
Base runtime	eclipse-temurin:17-jre-alpine (Alpine 3.23.4)
Data dos scans	2026-04-27 (UTC-3)
Banco	PostgreSQL 16-alpine (Docker Compose); H2 em testes
Superfície analisada	imagem (os-pkgs + JARs), código (Dockerfile + src/), dependências do fat-JAR (104 libs), working tree (segredos)

4.2 Ferramentas

Ferramenta	Versão	Alvo	Saída (em oficina/docs/security-scans/)
Trivy (image)	aquasec/trivy:latest	Imagem oficina-app:scan	trivy-image.{txt,json}
Trivy (fs)	idem	Filesystem do repo (vuln + secret + misconfig)	trivy-fs.txt
OWASP Dependency-Check	12.2.1 (NVD em 2026-04-27)	Fat-JAR (104 deps)	dependency-check-report.{html,json,xml,csv,sarif}
Gitleaks	zricethezav/gitleaks:latest	Working tree (--no-git)	gitleaks.json

4.3 Resultados automáticos

Trivy — imagem Docker:

Target	Tipo	CRITICAL/HIGH	Secrets
oficina-app:scan (alpine 3.23.4)	alpine	0	—
app/app.jar	jar	0	—

Trivy — filesystem: 1 misconfig **HIGH** — DS-0002 (Dockerfile sem USER non-root).

OWASP Dependency-Check — fat-JAR:

Métrica	Valor
Dependências analisadas	104

Métrica	Valor
Dependências com CVE	4
Severidade total	0 CRITICAL · 2 HIGH · 18 MEDIUM · 0 LOW

CVE	CVSSv3	Severidade	Componente	Análise
CVE-2018-1258	8.8	HIGH	spring-security-oauth2-resource-server-7.1.0-RC1 , spring-security-core-7.1.0-RC1	Falso positivo provável. CVE específico p/ Spring Security ≤ 5.0.4. Match decorre da heurística CPE do DC. Suprimir após validação manual.
CVE-2026-0540	6.1	MEDIUM	swagger-ui-5.20.1.jar (DOMPurify embarcado)	Bypass de allowlist via <code>USE_PROFILES</code> . Impacto restrito ao endpoint Swagger (não usado em produção pelo MVP).
CVE-2025-15599	6.1	MEDIUM	swagger-ui-5.20.1.jar	DOMPurify — bypass de URI validation com <code>ADD_ATTR</code> . Mesma mitigação.
CVE-2026-41240/41239/41238	—	MEDIUM	swagger-ui-5.20.1.jar	Família mXSS no DOMPurify. Mitigado por desativar SwaggerUI em produção.

Gitleaks: `gitleaks.json = []` . Zero matches. Arquivos `.env.example` e `docker-compose.yml` contêm apenas placeholders.

4.4 Revisão manual

#	Sev	Arquivo	Achado	Mitigação
M1	ALTA	SecurityConfiguration.kt	Usuários in-memory <code>atendente / tecnico / almoxarife</code> com senha = <code>username</code> ; admin tem default <code>admin</code> .	<code>UserDetailsService</code> com DB; senha aleatória na primeira subida; BCrypt strength ≥ 12.
M2	ALTA	JwtConfiguration.kt + docker-compose.yml	Default JWT secret fraco; sem validação de tamanho mínimo.	Falhar startup se <code>app.jwt.secret</code> ausente ou < 32 chars; secrets-manager em produção.
M3	MÉDIA	Dockerfile	Runtime executa como <code>root</code> (Trivy DS-0002 HIGH).	<code>RUN addgroup -S app && adduser -S app -G app + USER app</code> antes do <code>ENTRYPOINT</code> .
M4	MÉDIA	Dockerfile	<code>bootJar -x test</code> no build do container — testes ficam fora.	Estágio multi-stage <code>test</code> ou rodar testes em CI antes do <code>docker build</code> .
M5	MÉDIA	SecurityConfiguration.kt	<code>permitAll</code> em <code>/swagger-ui/**</code> , <code>/v3/api-docs/**</code> e <code>/actuator/health</code> .	Em produção: restringir Swagger por perfil (<code>@Profile("!prod")</code>); limitar <code>actuator</code> a <code>health,info</code> .

#	Sev	Arquivo	Achado	Mitigação
M6	MÉDIA	RestExceptionHandler.kt	MethodArgumentNotValidException expõe nomes de campos internos.	i18n em produção; logar detalhes server-side.
M7	BAIXA	application/api/dto/Dtos.kt	customerEmail / customerPhone sem regex/format.	Adicionar @Email e @Pattern (E.164/BR).
M8	BAIXA	Dtos.kt	LoginRequest sem rate-limit/lockout.	bucket4j com 5 tentativas/15 min em /api/public/auth/login .
M9	BAIXA	JwtIssuerService.kt	Fallback para SCOPE_ADMIN quando o usuário não tem authorities.	Substituir por IllegalStateException — evita <i>privilege escalation</i> .
M10	INFO	WorkOrderJpaRepository.kt	Queries usam JPQL com parameter binding.	Conforme. Sem SQL injection.
M11	INFO	SecurityConfiguration.kt	csrf().disable() + STATELESS + JWT.	Conforme. Padrão para REST API stateless.

4.5 Plano de mitigação priorizado

Prio	Item	Risco se ignorado	Esforço
P0	M1 — eliminar usuários demo / forçar senha aleatória	Acesso indevido em produção via senha trivial	M
P0	M2 — validar tamanho/origem do app.jwt.secret no startup	Forja de tokens HS256	P
P1	M3 — adicionar USER non-root no Dockerfile	Container escape em RCE	P
P1	M5 — perfil prod que desativa Swagger/Actuator detalhado	Info disclosure	P
P2	CVE-2026-0540 / CVE-2025-15599 — atualizar springdoc-openapi	XSS via Swagger UI	M
P2	CVE-2018-1258 — dependency-check-suppression.xml documentando o falso positivo	Ruído em scans futuros	P
P2	M4 — testes em CI antes do docker build	Regressões de segurança não barradas	M
P3	M6 — i18n de mensagens de validação	Info disclosure leve	M
P3	M7 — @Email / @Pattern em DTOs	Dados sujos	P
P3	M8 — rate-limit no /login	Brute force	M
P3	M9 — substituir fallback de scopes por exceção	Privilege escalation futuro	P

Esforço: P ($\leq 1h$), M (≤ 1 dia), G (> 1 dia).

4.6 Risco residual aceito (MVP — Fase 1)

- Imagem sem CVE CRITICAL/HIGH (Trivy 4.3).
- Único achado HIGH automático é misconfig do Dockerfile (DS-0002), tratável em < 30 min.

- Os 2 HIGH do DC (CVE-2018-1258) são falsos positivos confirmados — Spring Security 7.1.0-RC1 não é afetado.
- MEDIUMs concentrados em `swagger-ui-5.20.1` só impactam o endpoint Swagger, desativado em produção (P1/M5).
- Sem secrets vazados no working tree (Gitleaks).
- Queries JPA usam parameter binding — sem SQL injection observado.
- JWT HS256 com derivação SHA-256, dependente de segredo forte por env (P0/M2 endurece).

Para a Fase 2 (produção), os itens P0 e P1 acima são bloqueadores; P2/P3 entram em backlog regular.

4.7 Anexos brutos

Em `oficina/docs/security-scans/` : `trivy-image.{txt,json}` , `trivy-fs.txt` , `dependency-check-report.{html,json,xml,css,sarif}` , `dependency-check-jenkins.html` , `dependency-check-junit.xml` , `dependency-check-gitlab.json` , `gitleaks.json` , `oficina-fat.jar` (76 MB).